

第一单元：变量和数据类型

1. JavaScript 介绍

1.1. 历史背景

1.2. JavaScript和ECMAScript的关系

1.3. JavaScript 的作用

2. 页面引入 JavaScript

2.1. 行内方式

2.2. 标签方式

2.3. 外部链接方式

3. 定义变量和注释

3.1. 变量：存储单个可变信息的容器

3.2. 变量的命名规则

3.3. 三种输出方式

3.4. 注释

4. Javascript 数据类型

4.1. 数据类型

4.2. 判断类型：typeof

5. 数据类型转换

5.1. 其他类型转 String

5.2. 其他类型转 Number

5.3. 其他类型转 Boolean

6. 总结

1. JavaScript 介绍

1.1. 历史背景

布兰登·艾奇（Brendan Eich，1961年～），1995年在网景公司，发明的JavaScript。一开始JavaScript叫做LiveScript，但是由于当时Java这个语言特别火，所以为了傍大牌，就改名为JavaScript。

1.2. JavaScript和ECMAScript的关系

ECMAScript 是一种由Ecma国际前身为欧洲计算机制造商协会,英文名称是European Computer Manufacturers Association, 制定的标准。JavaScript是由公司开发而成的, 公司开发而成的一定是有一些问题, 不便于其他的公司拓展和使用。所以欧洲的这个ECMA的组织, 牵头制定JavaScript的标准, 取名为ECMAScript。

简单来说 ECMAScript 不是一门语言, 而是一个标准。符合这个标准的比较常见的有:

- **JavaScript: 浏览器端运行**
- **NodeJs: 服务器端运行**

1.3. JavaScript 的作用

1. 三层Web开发结构

- a. HTML: 结构层, 构成网页的骨架和内容
- b. CSS: 表现层, 定义网页的样式和布局
- c. JavaScript: 行为层, 负责网页的行为和交互性, 如操作DOM、响应事件

2. javascript 的功能

a. 与HTML的协作: 动态内容和交互性

- **动态内容:** JavaScript可以读取和修改HTML内容, 允许在网页加载后动态地改变文本、图片等元素。
- **用户交互:** 处理用户事件 (如点击、滚动、键盘输入), 并根据这些交互来更改页面内容或行为。

b. 与CSS的协作: 动态样式和动画

- **样式控制:** JavaScript能够实时更改元素的CSS样式, 使网页能够响应用户操作或其他事件来改变外观。
- **动画和视觉效果:** 使用JavaScript来创建复杂的动画和视觉效果, 这在纯CSS中难以实现。

3. javascript 组成部分 (重要)

- a. **ECMAScript 核心语法:** 主要规定了一些 javascript 的语法规范。
- b. **BOM:** 浏览器对象模型, 提供了访问浏览器的一些接口。
- c. **DOM:** 文档对象模型, 提供访问页面的接口。

2. 页面引入 JavaScript

2.1. 行内方式

行内示例

HTML

```
1 <button onclick="alert('hello world!')">按钮</button>
```

2.2. 标签方式

标签示例

HTML

```
1 <body>
2   <script>
3     alert('hello world!')
4   </script>
5 </body>
```

2.3. 外部链接方式

外部链接示例

HTML

```
1 <body>
2   <script src="index.js"></script>
3 </body>
```

3. 定义变量和注释

3.1. 变量：存储单个可变信息的容器

1. 变量分两部分

- 声明 `var a;` // 声明一个变量
- 赋值 `a = 100;` // 把100赋值给a

2. 声明和赋值同时进行：

- `var a = 100;` // 声明一个变量 a 并赋值为100;

3. 一次声明多个变量用逗号隔开；

- 例如：`var a = 100, b, c=30;`
- 这个示例同时声明了三个变量：**a**（赋值为100）、**b**（未赋值）、**c**（赋值为30）。

3.2. 变量的命名规则

1. 可包含字母、数字和下划线“_”和美元符“\$”。
2. 名称不能以数字开头，不能以汉字命名。
3. 不能使用 JavaScript 的关键字和保留字（如var、if、else、for等）。
4. 变量名严格区分大小写，例如，myname 和 myName 是两个不同的变量。

3.3. 三种输出方式

1. 打印在控制台: console.log(xxx);
2. 弹出框: alert(xxx);
3. 输出到页面: document.write(xxx);

3.4. 注释

- 注释的内容是不会被浏览器解析，是一个解释说明的信息，只是提供给程序员看

```
▼ javascript JavaScript |
1  // 单行注释
2  alert(100)
3  // alert(200) 此行代码不会被执行
4
5  /*
6     多行注释
7     alert(100)
8     alert(200)
9     alert(300)
10    此处代码都不会执行
11    */
12
```

4. Javascript 数据类型

4.1. 数据类型

1. 基础数据类型

1.1. number

- 整型
- 浮点型
- NaN
 - **not a Number**, 不是一个数值, 但他是number 类型
 - NaN表示一个本来要返回数字, 但是由于某种原因, 没有返回成功, 就用NaN表示
 - NaN与任何值都不行等, 包括其本身
 - NaN与其他任意值进行运算, 都为NaN

1.2. string

- 用引号引起来的就是字符串, 无论单引号还是双引号, 无论引号里面是什么
- 双引号和单引号必须要配对使用

▼ JavaScript |

```
1  // 正确写法
2  var str1 = "abc"
3  var str2 = 'ABC'
4
5  // 错误写法
6  var str = "abc'
```

1.3. boolean

- true
- false

1.4. undefined

1.5. null

2. 引用数据类型:

2.1. object

- {}
- []
- function

4.2. 判断类型: typeof

1. typeof 语法

▼ typeof JavaScript |

```
1 var a = 100
2 var b = 'abc'
3
4 console.log(typeof a) // 判断 a 的类型, 返回 'number'
5 console.log(typeof(b)) // 判断 b 的类型, 返回 'string'
```

2. 注意: typeof 本身的返回值是string类型的;

▼ typeof JavaScript |

```
1 var a = 100
2 var c = typeof a // c = 'number'
3
4 console.log(typeof c) // 返回 'string'
5 console.log(typeof typeof a) // 返回 'string'
```

3. 各数据类型的返回值 (重要)

- `typeof 123` => 'number'
- `typeof 'abc'` => 'string'
- `typeof true` => 'boolean'
- `typeof false` => 'boolean'
- `typeof undefined` => 'undefined'
- `typeof function(){}` => 'function'
- 注意: null、对象和数组使用 `typeof` 都是返回 'object' , 只有函数返回“function”
- `typeof null` => 'object'
- `typeof {}` => 'object'
- `typeof []` => 'object'

5. 数据类型转换

5.1. 其他类型转 String

1. 使用字符串拼接:

- 通过将任何类型的值与空字符串""拼接, 可以将其转换为字符串。
- 示例:

- `123 + ''` 返回 `"123"`
- `true + ''` 返回 `"true"`

2. 使用`toString()`方法:

- 大多数类型的值都可以使用`toString()`方法转换为字符串。
- 示例:
 - `(123).toString()` 返回 `"123"`
 - `(true).toString()` 返回 `"true"`
- 注意: `null`和`undefined`没有`toString()`方法, 尝试调用会导致错误。

3. 使用 `String()` 函数:

- `String()`函数可以将任何类型的值转换为字符串。
- 示例:
 - `String(123)` 返回 `"123"`
 - `String(null)` 返回 `"null"`
 - `String(undefined)` 返回 `"undefined"`

5.2. 其他类型转 Number

1. 字符串或数字转整型: `parseInt()`

- 从第一个字符开始转换, 如果第一个字符就是非数字, 那么得到的一定是`NaN`
- 如果第一个不是非数字, 就向后转换, 直到遇到非数字停止转换, 将前面转面转的内容输出

▼

parseInt

JavaScript

```
1 console.log(parseInt('3.14')) // 3
2 console.log(parseInt('3.14.5')) // 3
3 console.log(parseInt(3.14)) // 3
4 console.log(parseInt('123abc')) // 123
5 console.log(parseInt('abc123')) // NaN
```

2. 字符串转浮点数

- 从第一个字符开始转换, 如果第一个字符就是非数字, 那么得到的一定是`NaN`
- 如果第一个不是非数字, 就向后转换, 直到遇到第二个小数点或非数字, 则停止转换, 将前面转面转的内容输出

▼ parseFloat

JavaScript |

```
1 console.log(parseFloat("123.45")) // 123.45
2 console.log(parseFloat("123.45abc")) // 123.45
3 console.log(parseFloat("123.45.56.78")) // 123.45
4 console.log(parseFloat('abc123.45')) // NaN
```

3. 转数值

▼ Number

JavaScript |

```
1 // 可以转换成整型和浮点型
2 // 遇到不能转换的字符就返回NaN, 不论此字符在哪个位置都返回NaN
3 console.log(Number('123')) // 123
4 console.log(Number('123.45')) // 123.45
5 console.log(Number('123a')) // NaN
6 console.log(Number('123.45.67')) // NaN
7
8 // 特殊情况
9 console.log(Number('')) // 0
10 console.log(Number(null)) // 0
11 console.log(Number(true)) // 1
12 console.log(Number(false)) // 0
13 console.log(Number(undefined)) // NaN
```

4. isNaN(值): 判断变量是否为 NaN , 即检测某个变量是不是非数字

返回值: boolean [true->非数字 false->数字]

- a. 执行此函数时会先把值通过 Number 转成数字
- b. 再判断转换后的值是否为 NaN
- c. Object.is(val1, val2) 用于比较两个值是否严格相等, 类似于===操作符, 返回布尔值
- d. Number.isNaN() 和isNaN()的区别
 - i. isNaN() 在判断某个值是否为非数字之前, 会先隐式转换Number再判断
 - ii. Number.isNaN() 不会转换类型, 即 检查某个变量值是否严格等于NaN, 如果参数值不是NaN, 该方法都将返回false, 只有NaN返回true
 - iii. 使用场景: isNaN() 在判断前会先尝试转换数字, 可能会导致一些非预期结果, 因此, 在需要确保一个值确实是NaN时, 使用Number.isNaN()。如:排除非NaN的数字求和时

▼ isNaN

JavaScript |

```
1 console.log(isNaN(NaN)) // true
2 console.log(isNaN('123abc')) // true
3 console.log(isNaN('123')) // false
```

▼ Object.is()与===差异

JavaScript |

```
1 +0 === -0    // true
2 Object.is(+0, -0) // false
3
4 注意: NaN连自身都不相等, Object.is()解决了NaN与NaN不等的问题
5
6 NaN === NaN    // false
7 Object.is(NaN, NaN) // true
```

▼ Object.is()

JavaScript |

```
1 console.log(Object.is('foo', 'foo')); // true
2 console.log(Object.is({}, {})); // false, 对象比较的是引用
3 console.log(Object.is(+0, -0)); // false, +0 和 -0 被视为不相等
4 console.log(Object.is(-0, -0)); // true
5 console.log(Object.is(NaN, 0/0)); // true, NaN 与 NaN 被视为相等
6 console.log(Object.is(42, 42)); // true
7 console.log(Object.is(true, true)); // true
8 console.log(Object.is(false, false)); // true
9 console.log(Object.is(null, null)); // true
10 console.log(Object.is(undefined, undefined)); // true
```

▼ Number.isNaN与isNaN的区别

JavaScript |

```
1 console.log( isNaN("123") )    // Number("123") -> 123 // false
2 console.log( isNaN("hello") )  // Number("hello") -> NaN // true
3
4 console.log( Number.isNaN("123") ) // false
5 console.log( Number.isNaN("NaN") ) // false
6 console.log( Number.isNaN(NaN) )   // true
```

5.3. 其他类型转 Boolean

Boolean() 函数可以将任何类型的值转换为布尔值。在JavaScript中, 只有少数几个值会被转换为 **false**, 其余的都会被转换为 **true**。

以下值在转换时会变为布尔值 **false**:

1. 数值0和-0:

- 示例: `Boolean(0)` 和 `Boolean(-0)` 都返回 **false**

2. 空字符串:

- 示例: `Boolean("")` 返回 **false**

3. null:

- 示例: `Boolean(null)` 返回 **false**

4. undefined:

- 示例: `Boolean(undefined)` 返回 **false**

5. 数值NaN:

- 示例: `Boolean(NaN)` 返回 **false**

6. 布尔值false:

- 示例: `Boolean(false)` 返回 **false**

6. 总结

1. 了解 Javascript 和 ECMAScript 关系, 理解 JavaScript 的租用。
2. 页面引入 JS 的三种方式: 行内、标签内嵌、外部文件。
3. 定义变量的语法规则和规范, 和在页面输出变量的三种方式。
4. 单行注释和多行注释的语法。
5. JavaScript 数据类型: 基础类型和引用类型。
6. `typeof` 类型判断和它的返回值。
7. 各种数据类型的转换。